

mpc_cuda のアルゴリズム

正しく丸められる多倍長演算と、CUDA 向けレジスタ常駐

固定精度高速パス

tkouya

2026 年 6 月 18 日

概要

mpc_cuda は GNU 多倍長演算スタック—GMP (整数), MPFR (正しく丸められる 2 進浮動小数点), MPC (複素数)—を移植し、ホスト・デバイス双方の CUDA カーネル内で動作させる。本稿は、MPFR の `algorithms.tex` に倣って**アルゴリズム**を記述し、論文から引用できることを意図する。2 つの層を区別する。**実行時層** (`cu_mpfr_*`, `cu_mpc_*`) は上流 MPFR/MPC のアルゴリズムをそのまま再現し、アルゴリズム的な置き換えは 1 つだけ—`nvcc` 下で誤コンパイルされる MPFR の汎用除算パスを、`mpz` に基づく正しく丸められる除算で置き換える—である。**固定精度層**は、精度 p をコンパイル時定数とするレジスタ常駐型 `cu_freal<p>` と `cu_fcomplex<p>` を提供する。その基本算術は MPFR の最近接偶数丸め (RNDN) および MPC の成分ごとの MPC_RNDNN とビット単位で一致し、初等関数は**忠実** (*faithful*) (高精度, $\lesssim 1$ ULP, 正しく丸められるわけではない) である。格納モデル, 丸めの中核, スクールブック乗算, 複素乗算を正しく丸める厳密融合 `fmms/fmma`, および初等関数の作業精度 Newton 法/引数簡約方式を与える。さらにデバイス上で実行時層を高速化する要となるメモリ機構 (スレッドごとのバンプアリーナ) も記述する。

目次

1	範囲と 2 つの層	1
2	実行時層: MPFR/MPC からの逸脱	2
2.1	メモリ管理: スレッドごとのバンプアリーナ	2
2.2	デバイスで正しい除算	3
2.3	ホスト専用の関数	4
3	固定精度実数型 <code>cu_freal<p></code>	4
3.1	格納	4
3.2	丸めの中核 <code>finalize</code>	4
3.3	乗算	5
3.4	加算と減算	5
3.5	コスト	6
4	固定精度複素数型 <code>cu_fcomplex<p></code>	6
4.1	厳密融合 <code>fmms/fmma</code>	6
4.2	複素乗算	6
5	固定精度初等関数	7
5.1	Newton カーネル (除算・平方根・立方根)	7

5.2	超越関数：引数簡約と級数	7
5.3	複素初等関数	8
6	精度と検証	8
7	引用方法	9

1 範囲と 2 つの層

mpc_cuda は相互運用可能な 2 つの数値層を提供する。それぞれの丸め契約を表 1 にまとめる。

	実行時層	固定精度層
	cu_mpfr_t, cu_mpc_t	cu_freal<p>, cu_fcomplex<p>
精度	実行時（オブジェクトごと）	コンパイル時定数 p
格納	ヒープ／バンプアリーナ (limb 配列)	レジスタ ($\lceil p/64 \rceil$ limb)
丸めモード	MPFR/MPC 全モード (引数)	RNDN のみ (引数なし)
基本算術	上流 MPFR/MPC アルゴリズム	MPFR_RNDN / MPC_RNDNN とビット一致
除算	mpz 置換 (§2.2)	Newton 逆数 (§5)
初等関数	上流 MPFR/MPC (正しく丸め)	忠実, $\lesssim 1$ ULP (§5)

表 1 mpc_cuda の 2 つの数値層と丸め契約。

両層は同じ数値モデルを共有する。これを先に固定する。

■数値モデル. MPFR に従い、精度 p ビットの 0 でない有限数は

$$x = s \cdot m \cdot 2^{e-p}, \quad s \in \{-1, +1\}, \quad 2^{p-1} \leq m < 2^p, \quad m \in \mathbb{Z}, \quad (1)$$

すなわち符号、最上位ビット (MSB) が立った p ビット整数仮数 m 、および指数 e である。0 は別個のフラグ付き値である。仮数は $N = \lceil p/64 \rceil$ 個の 64 ビット limb に**左詰め** (MSB が最上位 limb の最上位ビット) で格納し、 $SB = 64N - p \in \{0, 32\}$ の下位ビットは 0 に保つ。これは MPFR のメモリ配置そのものであり、結果をホスト MPFR とビット単位で比較できる理由である。

以降の構成は次のとおり。実行時層と MPFR からの唯一のアルゴリズム的逸脱 (§2)、固定精度実数型 (§3)、固定精度複素数型とそれを正しく丸める厳密融合乗算 (§4)、固定精度初等関数 (§5)。§6 で精度保証と検証を、§7 で引用方法をまとめる。

2 実行時層：MPFR/MPC からの逸脱

実行時層は、上流 GMP/MPFR/MPC ソースから機械的移植 (cu_ 接頭辞付与、ホスト／デバイス修飾、ホスト専用 I/O の除去) で生成される。**アルゴリズム的には上流 MPFR/MPC そのもの**であり、加算・減算・乗算・平方根・定数・全超越関数は MPFR 自身のルーチンと Ziv の正しく丸めるループを用いる。したがってデバイス上でも、すべての丸めモードでホスト MPFR と同じビットを返す。複素層は MPC のままで、実部・虚部はパックされた丸めモード MPC_RND($r_{\Re}, r_{\Im}$) で独立に丸められる。引用のために記録すべき点は 2 つ：丸めの意味論が上流と同一であること、そして算術カーネルの置き換えがちょうど 1 つだけであること。さらに、実行時層を単に「正しい」だけでなく「高速」にする上で本質的なデバイス固有機構—メモリアロケータ (§2.1)—も記述に値する。

2.1 メモリ管理：スレッドごとのバンプアリーナ

すべての多倍長演算はメモリを確保する。mpfr_t は limb 用のバッファを必要とし、1 回の cu_mpfr_div や cu_mpc_mul は呼び出し連鎖を降りながら**数十個**の一時変数を確保・解放する。CPU では安価だが、デバイスの malloc/free は全スレッドが共有する単一のグローバル直列化アロケータであり、数千スレッドが競合すると支配的コストになる—本ライブラリでは、GPU が CPU と同等にとどまるか ~40× 高速になるかの分かれ目であった。これは純粋に実装上の関心事（結果は変わらない）だが、実行時層を大規模に使える理由なので、ここに記す。

mpc_cuda はデバイスヒープを**スレッドごとのバンプアリーナ**で置き換える。すなわち、1 つの連続した cudaMalloc 確保ブロックを、常駐スレッドごとに 1 つの固定サイズ**スラブ**へ分割し、グローバルスレッド ID で添字付けする（図 1）。3 つの操作は次のとおり。

- **確保 (Allocate)** = 要求サイズをスレッドのオフセット mpc_cuda_arena_top[tid] に加え、加える前の位置を返す。命令数個、探索もロックもなく、**スレッド間競合なし**（スレッドは互いのスラブに触れない）。
- **解放 (Free)** = **何もしない** (no-op)。
- **リセット (Reset)** = スレッドのオフセットを 0 に戻し、そのスレッドのスクラッチ領域全体を一括回収する。

解放が no-op なので、2 回のリセットの間に確保したものはすべて生存し続ける。すなわちアリーナは汎用ヒープではなく、**1 単位の作業のためのスクラッチパッド**である。カーネルは 1 つの作業項目に必要な確保をすべて行い、次の項目の前に mpc_cuda_arena_reset() を呼ぶ。同じフックが mini-GMP と、mpfr-gmp.c を介して MPFR・MPC の両方を裏付けるので、アリーナの導入は全層を一度に高速化する。サイズ設定によらず結果の正しさを保つ 2 つのフォールバックがある。アリーナ未導入 (mpc_cuda_arena_base = NULL) なら確保はデバイス malloc/free に向かう。作業項目がスラブを超過したら、**その確保は静かに malloc へあふれ**、通常どおり解放される。したがって超過は性能を損なうが、正しさは決して損なわない。

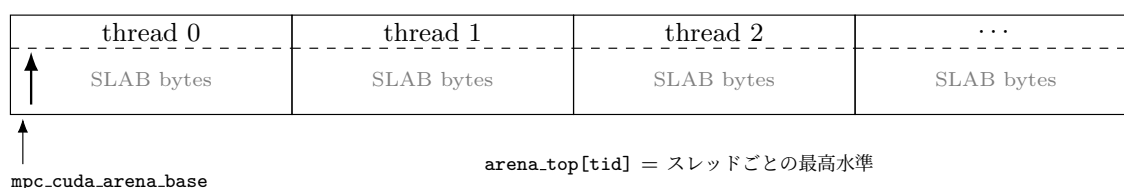


図 1 スレッドごとのバンプアリーナ：1 つの cudaMalloc ブロックを、常駐スレッドごとの固定サイズスラブに分割する。確保はスレッドのオフセットを上げ、解放は no-op、リセットは 0 に戻す。2 スレッドがスラブを共有しないため競合がない。

■**サイズ設定**. アリーナ総量は LAUNCH × SLAB である。LAUNCH は常駐スレッドの固定数（占有率のため選ぶ—深い呼び出し連鎖は limb をグローバルメモリのスラブに保持するので、その遅延を隠すには多数のワープが必要）、SLAB はスレッドごとのスクラッチで、1 作業項目の同時生存確保のピークを収める必要がある。解放が no-op なので、そのピークは最後のリセット以降に確保した総量に等しく、直接測定できる：mpc_cuda_arena_top[tid] が最高水準そのものなので、1 作業項目をリセットせずに実行すれば正確なバイト数が分かる。スクラッチは精度にほぼ線形にスケールし、経験的には（1024 ビットで）実数 MPFR の axpy / 内積はスレッドあたり ≈ 32 KB、複素数 MPC では ≈ 256 KB（約 8×、複素連鎖がより深い）を要する。過剰割り当てはメモリを費やすだけだが、過少割り当ては静かに低速パスへ落とす。

2.2 デバイスで正しい除算

MPFR の汎用除算 (`mpfr_div`, [4, 5] の分割統治／短除算パス) は, 3 limb 以上の精度に対して `nvcc` 下で誤コンパイルされ, デバイス上で `inf` やゴミを返す。原因は最適化器に絞り込まれたが, 問題の構文の特定には至らず, 一方で下位の構成要素 (`mpn_divrem`, mini-GMP の `mpz` 除算, `udiv_qrnnd`, `count_leading_zeros`) はいずれもデバイス上で**正しさを検証済み**である。そこで `mpc_cuda` は, デバイス上に限り, 信頼できる `mpz` プリミティブのみで構成した自己完結的で正しく丸められる除算を代わりに用いる (ホストは上流 MPFR を保持)。除算は定数と超越関数の基盤 (内部で除算する) なので, この置き換えが実行時層全体を解放する。

アルゴリズム 1 `mpfr_div_cuda(u, v, rnd)`: デバイスで正しい, 正しく丸められる除算

入力: 有限で 0 でない u, v , 精度は宛先 q (p ビット) 以下; 特異ケース ($0, \infty, \text{NaN}$) は MPFR と同様に別処理。

- 1: $s \leftarrow \text{sign}(u) \cdot \text{sign}(v)$
 - 2: u, v の仮数を整数 M_u, M_v (`mpz`) として構築
 - 3: $\text{shift} \leftarrow p + 2 + (v \text{ の limb 数}) \cdot 64$ ▷ 厳密な商 1 個分のガードビット
 - 4: $\text{num} \leftarrow M_u \ll \text{shift}$
 - 5: $(Q, R) \leftarrow \text{mpz_tdiv_qr}(\text{num}, M_v)$ ▷ 厳密な整数除算 1 回
 - 6: 丸めビットと sticky ビットを導出 ($\text{sticky} = [R \neq 0]$ と Q の落とした下位ビットの論理和)
 - 7: Q を p ビットに正規化, 商の指数を $e_u - e_v$ とシフトから設定
 - 8: 丸めモード rnd を $(Q, \text{round}, \text{sticky})$ に適用; 符号 s を設定
 - 9: **返す** `mpfr_check_range(q)` ▷ ホスト `mpfr_div` とビット一致
-

商は, 被除数を $p + 2$ ガードビット (さらに除数 1 個分) だけシフトしたものに対する単一の `mpz_tdiv_qr` で形成する (整数商に目標仮数・丸めビット・sticky 判定に十分な材料が既に含まれる)。続く丸めは**すべての**丸めモードについて MPFR を正確に踏襲するので, 公開関数 `cu_mpf_div` はホスト `mpfr_div` とビット一致する。コストは $O(p)$ limb の大整数除算 1 回で, MPFR のパスと漸近的に同クラスであり, 呼び出し側からは透過的である。

2.3 ホスト専用の関数

書式付き・文字列 I/O (`mpfr_printf`, `mpfr_get_str` のパース, `mpfr_set_str`, `mpz_out_str`, ...) と, mini-GMP がない `mpf_t/mpq_t` 変換はデバイスでは利用できない (デバイススタブは呼ばれるとトラップする)。これらはアルゴリズム的変更ではなく表層の制約だが, カーネルが呼べる範囲を画する。

3 固定精度実数型 `cu_freal<p>`

精度がコンパイル時に既知のとき, `cu_freal<p>` (ヘッダ `cu_freal.cuh`) は数全体をレジスタに保持し, アリーナと実行時精度ディスパッチを不要にする。 p は 32 の任意の正の倍数 (よって $SB \in \{0, 32\}$) であり, 仮数は $N = \lceil p/64 \rceil$ limb を占める。すべての演算が MPFR `RNDN` とビット一致する。

3.1 格納

値は (1) を実現する三つ組 (`sign`, `exp`, `m[N]`) である: `sign` $\in \{\pm 1\}$, `exp` は MPFR 指数, `m` は左詰めの仮数 (図 2)。予約指数で 0 を表す。配置が MPFR と一致するため, 同精度の実行時 `cu_mpf_t` との相互変換は単なるコピーである。

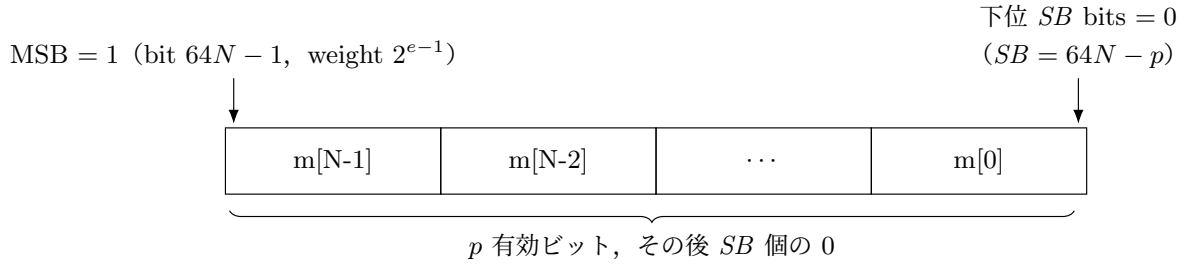


図2 `cu_frexp<p>` の左詰め仮数: N limb を上位から下位へ, MSB は常に立ち, 下位 SB ビットは 0 に固定。MPFR のメモリ配置と同一。

3.2 丸めの中核 finalize

すべての実数演算は 1 つのルーチン `finalize` に集約される。これは厳密な大きさ (MSB 位置と指数が既知のリトルエンディアン limb バッファ D と, D の下にある丸め/sticky 材料) を受け取り, `RNDN` で丸めた `cu_frexp<p>` を生成する。この単一の丸め点こそが, 本型が MPFR とビット一致することを保証する根拠である: 下の各カーネルは**厳密な**中間値を形成して `finalize` に渡すことだけに責任を負う。

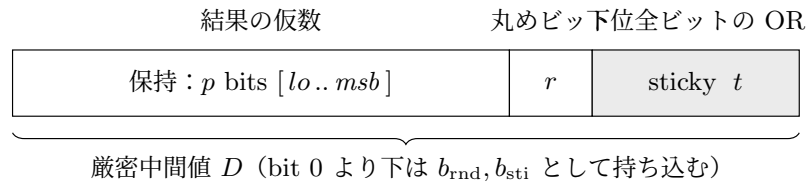


図3 `finalize` が用いる判定ビット: 保持する p ビット, そのすぐ下の丸めビット r , sticky ビット t (それより下のすべての論理和)。最近接偶数丸めは $r \wedge (t \vee \text{lsb})$ のとき切り上げる。

アルゴリズム 2 `finalize( $D, msb, E_{hi}, b_{rnd}, b_{sti}, s$ )`

- 1: $lo \leftarrow msb - p + 1$ ▷ D 内で保持する最下位ビット位置
 - 2: **もし** $lo > 0$ **ならば**
 - 3: $V \leftarrow D$ の bit [$lo..lo + p - 1$]; $r \leftarrow D_{lo-1}$; $t \leftarrow b_{rnd} \vee b_{sti} \vee (\bigvee_{i < lo-1} D_i)$
 - 4: **そうでなければ もし** $lo = 0$ **ならば**
 - 5: $V \leftarrow D$; $r \leftarrow b_{rnd}$; $t \leftarrow b_{sti}$
 - 6: **そうでなければ**
 - 7: $V \leftarrow D \ll (-lo)$; $r \leftarrow 0$; $t \leftarrow 0$ ▷ 有効ビットが p 未満: 厳密
 - 8: **終了 もし**
 - 9: **もし** $r \wedge (t \vee (V \bmod 2))$ **ならば** ▷ 最近接, タイは偶数へ
 - 10: $V \leftarrow V + 1$
 - 11: **もし** V が bit $p - 1$ から桁上がり **ならば**
 - 12: $V \leftarrow 2^{p-1}$; $E_{hi} \leftarrow E_{hi} + 1$
 - 13: **終了 もし**
 - 14: **終了 もし**
 - 15: $m \leftarrow V \ll SB$; $\text{exp} \leftarrow E_{hi} + 1$; $\text{sign} \leftarrow s$
 - 16: **返す** ($\text{sign}, \text{exp}, m$)
-

3つの分岐は, 通常ケース, 整列ケース, p 未満ビット (ゆえに厳密) ケースを扱う。桁上がり分岐は $1.\bar{1} \rightarrow 10.0$ の溢れを指数加算で処理する。double からの変換 (`from_double`) と double への変換 (`to_double`) も, 53

ビット IEEE 仮数上で同じ丸めビット／sticky ビット論理を用いる。

3.3 乗算

乗算は、レジスタ常駐のスクールブック $O(N^2)$ 乗算で**厳密な** $2N$ -limb 積を形成し (図 4), それを **finalize** で正規化・丸める。 p がコンパイル時定数なので二重ループは完全に展開され, **ptxas** は全 limb をレジスタに保持する。デバイスでは各 $64 \times 64 \rightarrow 128$ ステップが **mad.lo.cc.u64/madc.hi.u64** の対となり, 内積はグローバルメモリ通信なしで走る。

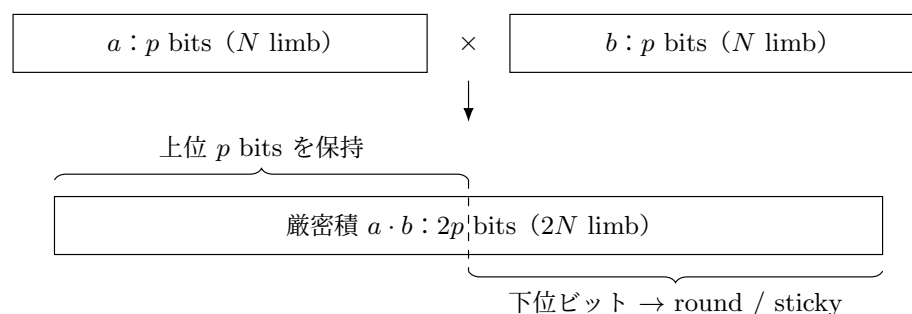


図 4 固定精度乗算：厳密な $2p$ ビット積をレジスタで計算し, 上位 p ビットを保持, 残りを **finalize** の round/sticky 判定に渡す。結果は $\circ(a \cdot b)$, すなわち p ビットの **mpfr_mul** に等しい。

3.4 加算と減算

加算は, 指数差 $d = e_a - e_b$ により小さい方のオペランドを整列し, limb 0 より下に押し出されたビットを round/sticky として捕捉し, $N+1$ -limb バッファに加算して丸める (図 5)。同符号の減算は, 2 個のガード limb (128 ビット余分) を持つバッファを用いるので, 激しい桁落ちでも正しく丸められた結果が残る：厳密差を形成し, 先頭の非零ビットを見つけ, そこから **finalize** が丸める。同符号／異符号の振り分けと大きさ比較は MPFR の **mpfr_add/mpfr_sub** を踏襲する。

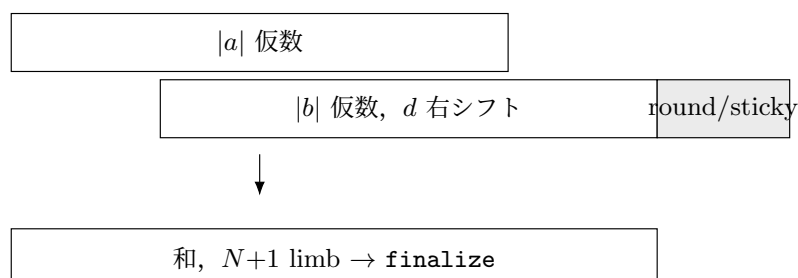


図 5 固定精度加算： $d = e_a - e_b$ で $|b|$ を整列, 押し出しビットを round/sticky として保持, 加算して一度丸める。減算は桁落ちを吸収する 2 個の余分なガード limb を用いる。

3.5 コスト

$N = \lceil p/64 \rceil$ として, 乗算は $O(N^2)$ 語乗算, 加減算は $O(N)$ で, いずれもレジスタ常駐・確保なし。スクールブック乗算は GMP/MPFR の劣 2 次乗算より漸的に劣るので, 固定精度の優位は低・中精度 (limb をレジスタに保つことが支配的) で最大となり, 高 p で作業集合がレジスタから溢れると縮む。

4 固定精度複素数型 `cu_fcomplex<p>`

`cu_fcomplex<p>` (ヘッダ `cu_fcomplex.cuh`) は `cu_freal<p>` 成分の対である。加減算は成分ごとの `cu_fadd/cu_fsub` なので自明に MPC_RNDNN。興味深いのは乗算で、二重丸めを避けることで**成分ごとに正しく丸められる**—MPC の MPC_RNDNN とビット一致する。

4.1 厳密融合 `fmms/fmma`

素朴な複素積 $\Re = ar br - ai bi$, $\Im = ar bi + ai br$ は、4 つの実積を減算／加算の前にそれぞれ丸めるため、成分あたり 2 回丸めが入り MPC と一致しない。代わりに `mpc_cuda` は各成分を融合プリミティブ

$$\text{fmms}(a, b, c, d) = \circ(ab - cd), \quad \text{fmma}(a, b, c, d) = \circ(ab + cd), \quad (2)$$

で評価する。各々は 2 つの**厳密な** $2N$ -limb 積を**単一**の RNDN 丸めで合成する (図 6)。具体的には各積 ab と cd を §3.3 のスクールブック乗算で厳密に形成し、符号付き広項 (`sign, exp, M[2N]`) に正規化する。次にこの 2 つの広項を指数差で整列 (3 個のガード limb 付き) して厳密に加減算し、`finalize` で一度だけ丸める。よって結果は中間丸めなしの $\circ(ab \pm cd)$ に等しく、これはまさに MPFR の `mpfr_fmms/mpfr_fmma` (ひいては MPC) が計算するものである。

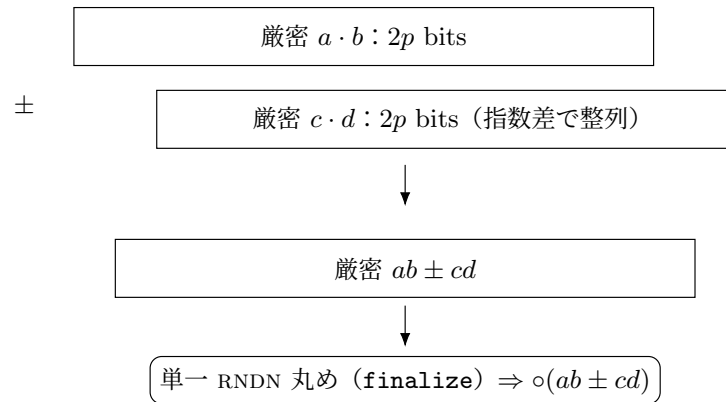


図 6 厳密融合 `fmms/fmma`: 両 $2p$ ビット積を厳密に保ち、整列・合成し、**一度だけ**丸める。二重丸めがないので、各成分は MPC の MPC_RNDNN とビット一致する。

4.2 複素乗算

融合プリミティブを使うと、複素乗算は成分厳密で評価された教科書的恒等式となる：

$$(ar + i ai)(br + i bi) = \underbrace{\text{fmms}(ar, br, ai, bi)}_{\Re = \circ(ar br - ai bi)} + i \underbrace{\text{fmma}(ar, bi, ai, br)}_{\Im = \circ(ar bi + ai br)}. \quad (3)$$

これは「 $3M$ 不使用」の精度優先の定式化であり、4 つの厳密積と 2 つの単丸め合成からなる。 $p = 32 \dots 2048$ にわたり MPC とビット一致を検証済みである。

5 固定精度初等関数

初等関数 (実数はヘッダ `cu_fmath.cuh`, 複素数は `cu_fcmath.cuh`) は、正しい丸めをレジスタ常駐性と引き換えにする。各関数は**作業精度**

$$W = p + g, \quad g = \text{CU_FGUARD} = 128 \text{ bits (既定)}, \quad (4)$$

で, `cu_freal<W>` 上の Newton 反復と引数簡約級数により評価し, W ビット結果を p へ丸め直す。結果はおよそ p ビットに**忠実**で, 一般的な範囲で MPFR/MPC と $\lesssim 1$ ULP 一致する。これらは意図的に**正しく丸められない**: テーブルメーカーのジレンマには無制限の Ziv ループが必要で, 固定レジスタ予算と相容れない。最後の 1 ビットまでの正しさが必要なら, 実行時 `cu_mpfr/cu_mpc` API を用いる。

5.1 Newton カーネル（除算・平方根・立方根）

各代数関数は `double` 近似で初期化した自己修正 Newton 写像を反復する。反復回数は $\lceil \log_2(W/52) \rceil + 1$ で, 各ステップごとに正しい桁数が倍化する (表 2)。

関数	量	Newton 更新	復元
<code>cu_fdiv</code>	逆数 $z \approx 1/m$	$z \leftarrow z(2 - mz)$	$a/b = a \cdot (1/b)$
<code>cu_sqrt</code>	rsqrt $y \approx 1/\sqrt{m}$	$y \leftarrow \frac{1}{2}y(3 - my^2)$	$\sqrt{m} = m \cdot y$
<code>cu_cbrt</code>	rcbrt $y \approx m^{-1/3}$	$y \leftarrow y(4 - my^3)/3$	$\sqrt[3]{m} = m \cdot y^2$

表 2 固定精度代数関数を用いる除算なし Newton 反復。 m は 2 進スケーリングで固定区間に収めた仮数で, 2 のべきは後で付け直す。

逆数反復は乗算と減算のみ（除算なし）を用いる。これが `cu_fdiv` すらハードウェア除算を避ける理由である。`cu_div_ui` (小さな $< 2^{32}$ 整数による除算) は, 級数内で用いる別個の正しく丸められる limb 単位ルーチンである。

5.2 超越関数：引数簡約と級数

超越関数は引数を小区間へ簡約し, Taylor (あるいは `atanh/atan`) 級数を項が作業精度の閾値を下回るまで加算し, 復元する。方式を表 3 にまとめる。定数 $\ln 2$ と π 自体も作業精度で, $2 \operatorname{atanh}(1/3)$ と Machin の $16 \arctan(1/5) - 4 \arctan(1/239)$ により計算する。

関数	簡約	級数／復元
<code>cu_exp</code>	$x = k \ln 2 + r$, $k = \lfloor x / \ln 2 \rfloor$; $r \rightarrow r/2^m$ と半減	$\exp(r) = \sum r^n/n!$, m 回 2 乗, 2^k 倍
<code>cu_log</code>	$x = 2^e m$, $m \in [\frac{1}{\sqrt{2}}, \sqrt{2})$; $t = \frac{m-1}{m+1}$	$\log m = 2 \operatorname{atanh}(t)$, $e \ln 2$ を加算
<code>cu_sin/cos</code>	$x = k \frac{\pi}{2} + r$, $ r \leq \frac{\pi}{4}$; r を半減	$\sin r, \cos r$ の Taylor, 倍角で復元, $k \bmod 4$ で象限選択
<code>cu_atan</code>	$1/x$ で $ x \leq 1$ に簡約; $a \leftarrow a/(1 + \sqrt{1 + a^2})$	$\arctan$ 級数, $2^{(\text{簡約回数})}$ 倍
<code>cu_asin/acos</code>	$\arcsin x = \arctan(x/\sqrt{1-x^2})$; $\arccos = \pi/2 - \arcsin$	<code>cu_atan</code> 経由
<code>cu_sinh/cosh</code>	e^x 経由 (sinh は小 x で Taylor)	$\frac{1}{2}(e^x \mp e^{-x})$
<code>cu_atanh</code>	$ x \leq \frac{1}{2}$ は級数; 他は $\frac{1}{2} \log \frac{1+x}{1-x}$	atanh 級数 / <code>cu_log</code>
<code>cu_pow</code>	$x^y = \exp(y \log x)$ ($x > 0$); $x < 0$ は整数 y 分岐	<code>cu_exp</code> , <code>cu_log</code> 経由

表 3 固定精度超越関数の引数簡約と復元。全中間値は $W = p + 128$ の `cu_freal<W>`。

5.3 複素初等関数

複素層 (`cu_fcmath.cuh:cu_cexp`, `cu_clog`, `cu_csqrt`, `cu_csin`, `cu_ccos`, `cu_ctan`, `cu_csinh`, `cu_ccosh`, `cu_cdiv`, および `atan2`) は, 実数関数に加え, すべての絶対値 2 乗と交差項に §4.1 の**厳密**な `fmms` / `fmma` を用いて構築する。例えば

$$|z|^2 = \text{fmma}(\Re z, \Re z, \Im z, \Im z), \quad \log z = \frac{1}{2} \log |z|^2 + i \operatorname{atan2}(\Im z, \Re z), \quad (5)$$

$$e^z = e^{\Re z} (\cos \Im z + i \sin \Im z), \quad \frac{a}{b} = \frac{\text{fmma}(ar, br, ai, bi) + i \text{fmms}(ai, br, ar, bi)}{\text{fmma}(br, br, bi, bi)}. \quad (6)$$

$|b|^2$ と分子に厳密融合積を用いることで, 複素除算と絶対値が二重丸めで精度を失うのを防ぐ。実数版と同様, これらも忠実で MPC に対し $\lesssim 1 \text{ ULP}$ である。

6 精度と検証

演算	保証	検証範囲
実行時 <code>cu_mpfr/cu_mpc</code> (<code>cu_mpfr_div</code> 含む)	全モードで正しく丸め; ホストと ビット一致	ホスト MPFR/MPC と一致
<code>cu_freal</code> $+$, $-$, $\times$	MPFR <code>RNDN</code> とビット一致	$p = 32 \dots 2048$
<code>cu_fcomplex</code> $+$, $-$, $\times$	MPC <code>RNDNN</code> とビット一致	$p = 32 \dots 2048$
<code>cu_fmath/cu_fcmath</code>	忠実, $\lesssim 1 \text{ ULP}$ (正しく丸めない)	$p = 64 \dots 1024$

表 4 演算ごとの精度契約と検証済み精度範囲。

基本算術の 2 つの主張はシステム MPFR・MPC とビット単位で照合し, 初等関数の主張は MPFR/MPC に対し ULP 単位で照合する (同梱スイートは標本入力で 0 ULP を報告し, 設計上の上界は $\leq 1 \text{ ULP}$)。契約を表 4 にまとめる。

7 引用方法

本稿は `mpc_cuda` のアルゴリズムを記述する。引用される可能性が最も高い結果は, (i) 単一の丸め点と厳密融合 `fmms` / `fmma` により MPFR `RNDN` と MPC `MPC.RNDNN` にビット一致する, レジスタ常駐の固定精度算術 (§3–4), および (ii) デバイス上で MPFR の汎用パスを置き換える `mpz` ベースの正しく丸められる除算 (§2.2), の 2 つである。性能評価 (GPU/CPU クロスオーバーや, 1024 ビット AXPY における `cu_freal` の実行時 GPU パスに対する $\sim 140\times$ 高速化など) は付属の技術報告 (`mpc_cuda_report`) に与える。実行時層の正しく丸める意味論は MPFR [2] と MPC [3] に従い, これらは GMP [1] の上に構築される。基礎となる手法は標準的である [6, 7]。

参考文献

- [1] T. Granlund and the GMP development team, *GNU MP: The GNU Multiple Precision Arithmetic Library*. <https://gmplib.org/>.
- [2] L. Fousse, G. Hanrot, V. Lefèvre, P. Pélicissier, P. Zimmermann, “MPFR: A multiple-precision binary floating-point library with correct rounding,” *ACM Trans. Math. Softw.*, 33(2), 2007. <https://www.mpfr.org/>.

- [3] A. Enge, M. Gastineau, P. Théveny, P. Zimmermann, *GNU MPC: A library for multiprecision complex arithmetic with exact rounding*. <https://www.multiprecision.org/mpc/>.
- [4] D. Harvey and P. Zimmermann, “Short Division of Long Integers,” *Proc. 20th IEEE Symp. on Computer Arithmetic (ARITH-20)*, 2011, pp. 7–14.
- [5] N. Möller and T. Granlund, “Improved Division by Invariant Integers,” *IEEE Trans. Computers*, 60(2):165–175, 2011.
- [6] R. P. Brent and P. Zimmermann, *Modern Computer Arithmetic*, Cambridge University Press, 2010.
- [7] J.-M. Muller et al., *Handbook of Floating-Point Arithmetic*, 2nd ed., Birkhäuser, 2018.